

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO BÀI TẬP LỚN
CÁC HỆ THỐNG PHÂN TÁN

**Đề tài : Trình soạn thảo tài liệu cộng tác thời
gian thực**

Nhóm 20

Giảng viên : TS. Kim Ngọc Bách

Thành Viên :

Bùi Đức Huy – B22DCCN375

Nguyễn Trung Hiếu – B22DCCN315

Trần Đức Hoàng Lâm – B22DCCN483

Hà Nội – 2026

Mục lục

I. KIẾN TRÚC HỆ THỐNG	4
1.1. Tổng quan và mục tiêu thiết kế	4
1.2. Sơ đồ kiến trúc tổng thể.....	5
1.3. Tầng Client (Frontend)	5
1.4. Tầng backend và dịch vụ thời gian thực.....	6
1.5. Tầng lưu trữ	7
1.6. Mô hình phân quyền và bảo mật	7
1.7. Luồng xác thực và thiết lập kết nối WebSocket.....	8
II. MÔ HÌNH ĐỒNG BỘ DỮ LIỆU	9
2.1. Bài toán đồng bộ trong hệ phân tán.....	9
2.2. CRDT và lý do chọn Liveblocks/Yjs	9
2.3. Nguyên lý CRDT qua ví dụ.....	9
2.4. Giao thức đồng bộ	10
2.5. Trình tự đồng bộ thời gian thực đầu-cuối.....	11
2.6. Lưu trữ và khôi phục trạng thái	11
2.6.1. Luồng đọc khi mở tài liệu.....	11
2.6.2. Luồng ghi khi chỉnh sửa	12
2.7. Đồng bộ room storage	12
2.8. Đồng bộ presence, cursor và comments	12
2.9. Tính nhất quán cuối và các đảm bảo của hệ thống.....	12
III. GIAO THỨC TRAO ĐỔI DỮ LIỆU	13
3.1. Giao thức HTTP kết hợp Convex client.....	13
3.1.1. Giới thiệu và định nghĩa	13
3.1.2. Cách thức áp dụng trong dự án.....	13
3.1.3. Định dạng dữ liệu và bảo mật.....	13
3.1.4. Lý do lựa chọn sử dụng	14
3.1.5. Đánh giá ưu và nhược điểm.....	14
3.2. Giao thức WebSocket với Liveblocks.....	14
3.2.1. Giới thiệu và định nghĩa	14
3.2.2. Cách thức áp dụng trong dự án.....	14
3.2.3. Định dạng dữ liệu và bảo mật.....	14

3.2.4. Lý do lựa chọn sử dụng	14
3.2.5. Đánh giá ưu và nhược điểm.....	15
3.3. Giao thức đồng bộ CRDT/Yjs.....	15
3.3.1. Giới thiệu và định nghĩa	15
3.3.2. Cách thức áp dụng trong dự án.....	15
3.3.3. Định dạng dữ liệu và bảo mật.....	15
3.3.4. Lý do lựa chọn sử dụng	15
3.3.5. Đánh giá ưu và nhược điểm.....	15
3.4. Giao thức Awareness/Presence	16
3.4.1. Giới thiệu và định nghĩa	16
3.4.2. Cách thức áp dụng trong dự án.....	16
3.4.3. Định dạng dữ liệu và bảo mật.....	16
3.4.4. Lý do lựa chọn sử dụng	16
3.4.5. Đánh giá ưu và nhược điểm.....	16
IV. THỬ NGHIỆM VÀ ĐÁNH GIÁ	17
4.1. Giao diện đăng nhập và xác thực.....	17
4.2. Giao diện trang chủ.....	17
4.3. Tạo và quản lý tài liệu	18
4.4. Soạn thảo rich text	21
4.5. Cộng tác thời gian thực.....	22
4.6. Bình luận và quản lý người dùng đang chỉnh sửa	23
4.7. Xuất tài liệu và undo/redo	24
4.8. Đánh giá chung.....	25
4.9. Hướng phát triển.....	25
KẾT LUẬN	25

I. KIẾN TRÚC HỆ THỐNG

1.1. Tổng quan và mục tiêu thiết kế

Hệ thống soạn thảo văn bản cộng tác thời gian thực là một ứng dụng web cho phép nhiều người dùng cùng tạo, chỉnh sửa và trao đổi trên cùng một tài liệu. Về bản chất, đây là một hệ phân tán vì trạng thái tài liệu được sao chép và cập nhật đồng thời tại nhiều client, trong khi dữ liệu định danh, metadata và luồng đồng bộ được xử lý bởi các dịch vụ backend khác nhau.

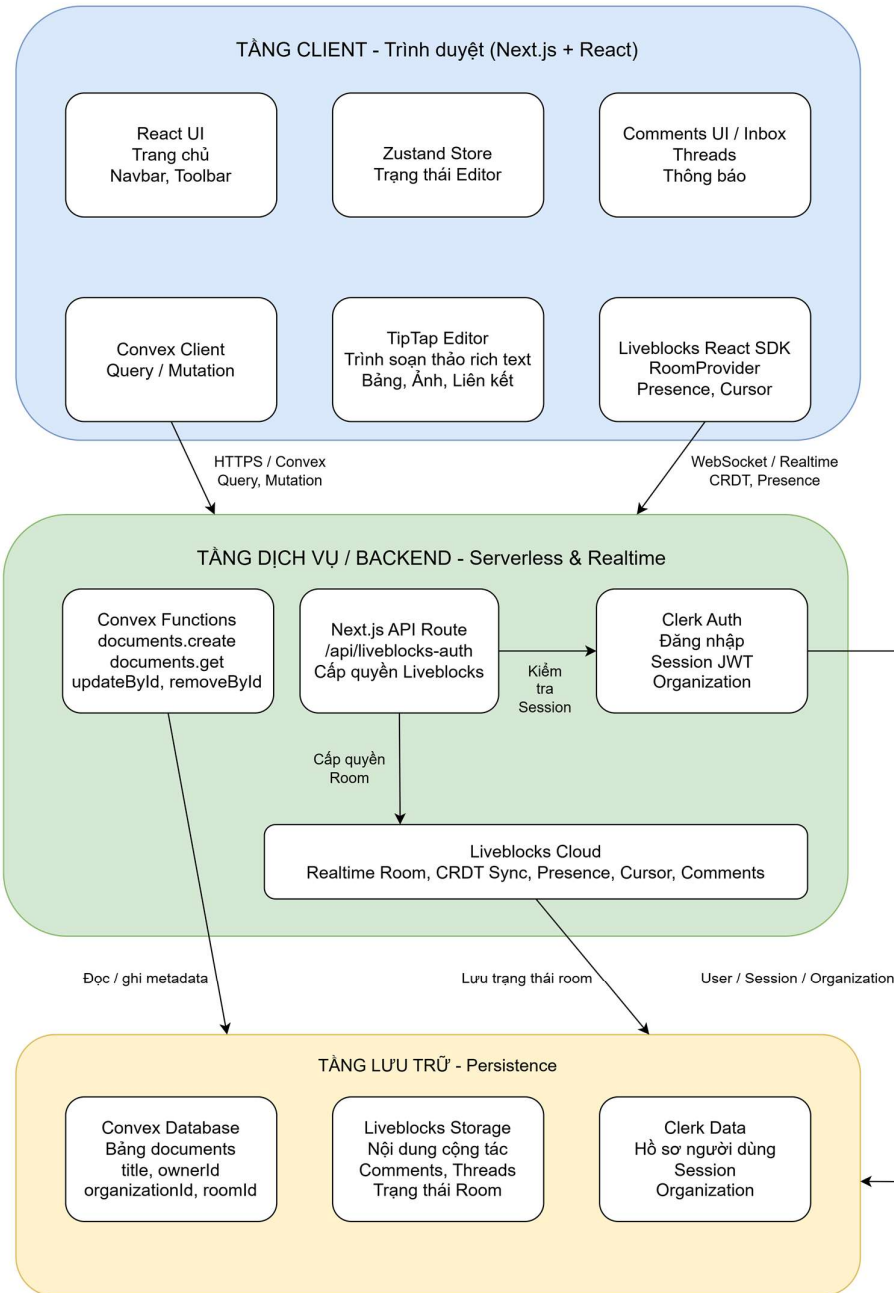
Dự án được xây dựng theo hướng tương tự Google Docs ở phạm vi bài tập lớn: người dùng đăng nhập, tạo tài liệu, mở tài liệu trong trình soạn thảo rich text, quan sát người đang online, bình luận và chỉnh sửa gần như tức thời với các client khác.

Kiến trúc hướng tới bốn mục tiêu chính:

- **Thời gian thực:** thao tác của một người dùng cần được truyền tới những người còn lại với độ trễ thấp.
- **Không mất chỉnh sửa đồng thời:** các thao tác nhập, xóa và định dạng văn bản phải được hợp nhất thay vì ghi đè đơn giản.
- **Phân tách trách nhiệm:** xác thực, lưu metadata, render giao diện và đồng bộ realtime được đặt ở các thành phần chuyên trách.
- **Dễ mở rộng:** frontend, backend serverless và dịch vụ realtime có thể mở rộng độc lập theo tải sử dụng.

Hệ thống sử dụng Next.js và React cho giao diện, TipTap/ProseMirror cho trình soạn thảo, Liveblocks cho đồng bộ thời gian thực, Convex cho dữ liệu nghiệp vụ, Clerk cho xác thực và Zustand cho trạng thái editor phía client.

1.2. Sơ đồ kiến trúc tổng thể



Hình 1. Kiến trúc tổng thể của hệ thống soạn thảo cộng tác.

Điểm quan trọng của kiến trúc là client làm việc song song với hai luồng dữ liệu: luồng nghiệp vụ qua Convex để quản lý danh sách tài liệu, tiêu đề, chủ sở hữu và phân trang; luồng realtime qua Liveblocks để đồng bộ nội dung, con trỏ, presence, comments và shared storage trong từng room.

1.3. Tầng Client (Frontend)

Client là ứng dụng Next.js App Router. Các màn hình chính nằm trong `src/app`:

Khu vực	Vai trò	Thành phần tiêu biểu
Trang chủ	Hiển thị danh sách tài liệu, template, tìm kiếm, tạo mới	src/app/(home)/page.tsx, documents-table.tsx, templates-gallery.tsx
Trang tài liệu	Cung cấp trình soạn thảo, toolbar, ruler, navbar, avatar, comments	src/app/documents/[documentId]/document.tsx, editor.tsx, toolbar.tsx
API xác thực realtime	Cấp quyền truy cập Liveblocks room	src/app/api/liveblocks-auth/route.ts
Provider toàn cục	Kết nối Convex với Clerk và React tree	src/components/convex-client-provider.tsx

Trình soạn thảo sử dụng TipTap, một framework editor dựa trên ProseMirror. Component Editor khởi tạo các extension cần thiết cho bài toán soạn thảo văn bản:

- StarterKit cho paragraph, heading, danh sách, blockquote và các node cơ bản.
- Table, TableRow, TableCell và TableHeader cho soạn thảo bảng.
- Image và resize image cho chèn hình ảnh.
- TextStyle, Color, Highlight, FontFamily, FontSize và LineHeight cho định dạng văn bản.
- TextAlign, Underline và Link cho các thao tác văn phòng phổ biến.
- Liveblocks TipTap extension để biến các thay đổi của editor thành dữ liệu cộng tác realtime.

Hai extension tự xây dựng gồm src/extensions/font-size.ts và src/extensions/line-height.ts, giúp toolbar điều khiển cỡ chữ và giãn dòng trực tiếp trên paragraph/heading. Trạng thái editor được chia sẻ qua Zustand trong src/store/use-editor-store.ts để toolbar và các thành phần khác có thể thao tác trên cùng một instance editor.

1.4. Tầng backend và dịch vụ thời gian thực

Backend của hệ thống không được triển khai như một server đơn khối, mà kết hợp các dịch vụ serverless/chuyên dụng:

1. **Convex:** chịu trách nhiệm lưu metadata tài liệu, cung cấp query/mutation, phân trang và tìm kiếm.
2. **Liveblocks:** chịu trách nhiệm đồng bộ realtime cho từng room, bao gồm nội dung CRDT, presence, cursor, comments và storage dùng chung.
3. **Clerk:** chịu trách nhiệm đăng nhập, session, thông tin người dùng và organization.
4. **Next.js API route:** đóng vai trò cầu nối bảo mật khi client cần xác thực để vào Liveblocks room.

Thiết kế này tương đương mô hình hai luồng server song song trong mẫu báo cáo: một luồng request/response cho nghiệp vụ ít thay đổi, một luồng WebSocket duy trì liên tục cho đồng bộ

cộng tác. Việc tách luồng giúp thao tác realtime không làm nghẽn các API quản lý tài liệu và ngược lại.

1.5. Tầng lưu trữ

Tầng lưu trữ được chia theo loại dữ liệu:

Nhóm dữ liệu	Nơi lưu trữ/xử lý	Mục đích
Metadata tài liệu	Convex DB	Tiêu đề, chủ sở hữu, organization, room id, nội dung khởi tạo
Nội dung cộng tác	Liveblocks room/storage	Dữ liệu rich text được đồng bộ realtime qua CRDT
Presence và cursor	Liveblocks presence	Người đang online, vùng chọn, màu đại diện
Comments và threads	Liveblocks comments	Trao đổi theo đoạn văn bản trong tài liệu
Thông tin định danh	Clerk	User id, session, organization id, JWT cho Convex
Trạng thái UI editor	Zustand phía client	Instance editor, toolbar state

Trong Convex, bảng *documents* được định nghĩa trong *convex/schema.ts*. Các trường chính gồm *title*, *initialContent*, *ownerId*, *roomId* và *organizationId*. Các thao tác nghiệp vụ được đặt trong *convex/documents.ts*.

1.6. Mô hình phân quyền và bảo mật

Hệ thống dựa vào Clerk để xác thực người dùng. Khi gọi Convex, client lấy identity từ Clerk thông qua *ConvexProviderWithClerk*. Ở phía Convex, các query/mutation dùng *ctx.auth.getUserIdentity()* để xác định người đang thao tác.

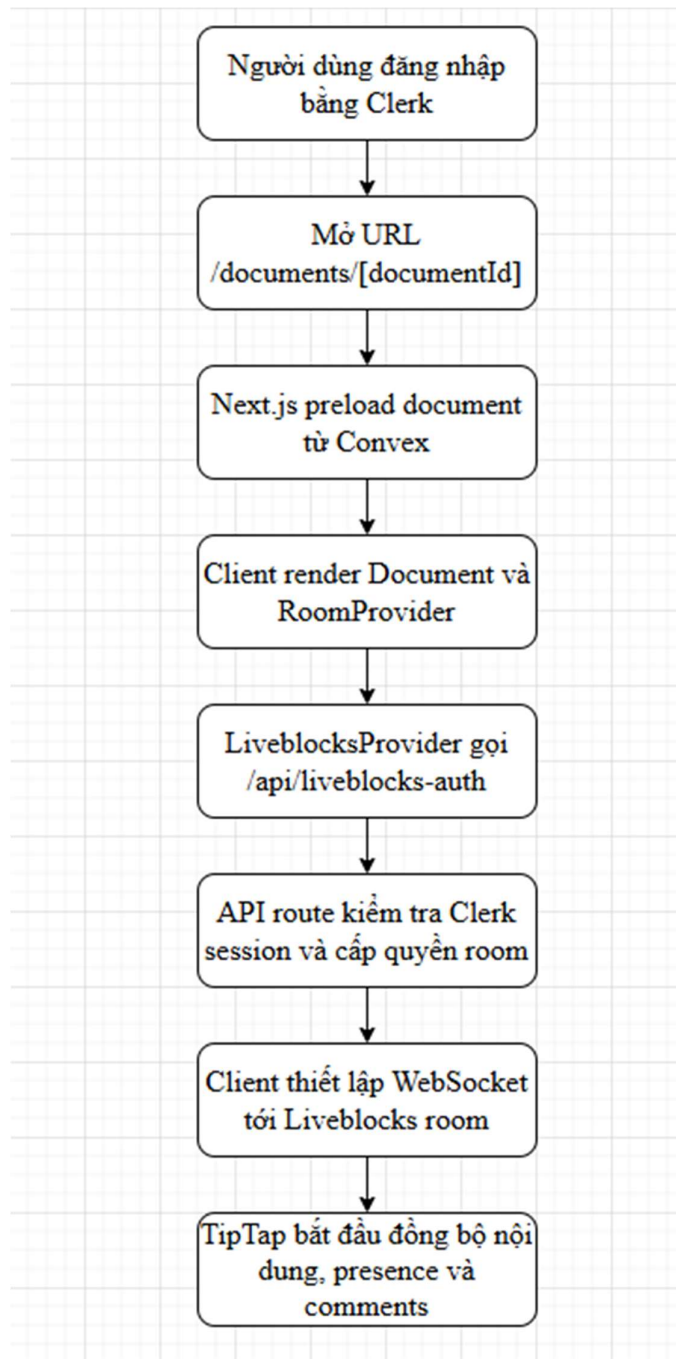
Cách kiểm soát quyền hiện tại tập trung vào chủ sở hữu và organization:

- Tài liệu được gắn với *ownerId* là id người tạo.
- Nếu người dùng đang ở organization, document có thể gắn với *organizationId*.
- Các thao tác sửa/xóa/lấy chi tiết kiểm tra identity trước khi thực hiện.
- Nếu không có identity hợp lệ, hệ thống trả lỗi *Unauthorized*.

Đối với Liveblocks, client không tự ý mở room trực tiếp bằng secret key. Thay vào đó, client gọi API route */api/liveblocks-auth*. Route này kiểm tra session Clerk ở server side, lấy thông tin tài liệu từ Convex nếu cần, sau đó tạo session Liveblocks với quyền truy cập room tương ứng.

1.7. Luồng xác thực và thiết lập kết nối WebSocket

Luồng mở tài liệu và tham gia realtime room diễn ra như sau:



Hình 2. Trình tự xác thực và thiết lập kết nối realtime.

Luồng này bảo đảm secret key của Liveblocks chỉ tồn tại ở phía server. Client chỉ nhận session đã được cấp quyền, phù hợp với nguyên tắc không đưa khóa bí mật vào trình duyệt.

II. MÔ HÌNH ĐỒNG BỘ DỮ LIỆU

2.1. Bài toán đồng bộ trong hệ phân tán

Trong trình soạn thảo cộng tác, nhiều người dùng có thể cùng sửa một đoạn văn bản tại cùng thời điểm. Nếu hệ thống dùng cơ chế ghi sau đề ghi trước, thao tác của người dùng này có thể làm mất thao tác của người dùng khác. Nếu dùng khóa tài liệu, chỉ một người được sửa tại một thời điểm, làm mất ý nghĩa cộng tác thời gian thực.

Vì vậy, hệ thống cần một cơ chế đồng bộ có các đặc điểm:

- Cho phép mỗi client chỉnh sửa cục bộ ngay lập tức.
- Hợp nhất các thao tác đồng thời mà không cần khóa bí quan.
- Đưa tất cả client về cùng một trạng thái cuối.
- Chịu được độ trễ mạng và thứ tự nhận message khác nhau.

Dự án giải quyết bài toán này bằng CRDT thông qua Liveblocks/Yjs và tích hợp trực tiếp với TipTap.

2.2. CRDT và lý do chọn Liveblocks/Yjs

CRDT là viết tắt của Conflict-free Replicated Data Type. Đây là nhóm cấu trúc dữ liệu cho phép nhiều bản sao được cập nhật độc lập, sau đó tự hội tụ về cùng trạng thái khi các update được trao đổi đầy đủ.

Lý do chọn Liveblocks/Yjs cho dự án:

- **Phù hợp editor rich text:** Yjs có hệ sinh thái tốt với ProseMirror/TipTap.
- **Hỗ trợ cộng tác đầy đủ:** Liveblocks không chỉ đồng bộ nội dung mà còn cung cấp presence, cursor, comments và room storage.
- **Giảm tải tự vận hành server realtime:** dự án không cần tự triển khai WebSocket server, persistence và fan-out message.
- **Tích hợp React tốt:** `@liveblocks/react`, `@liveblocks/react-tiptap` và `@liveblocks/react-ui` giúp giảm lượng mã kết nối.

So với mô hình tự triển khai Hocuspocus/WebSocket, Liveblocks giúp hệ thống đơn giản hơn ở tầng vận hành nhưng phụ thuộc nhiều hơn vào dịch vụ cloud bên ngoài.

2.3. Nguyên lý CRDT qua ví dụ

Giả sử nội dung ban đầu là:

Hệ thống phân tán

Hai người dùng thao tác cùng lúc:

User A chèn "môn " trước "phân tán"
User B chèn "PTIT " trước "phân tán"

Nếu dùng ghi đè thông thường, một trong hai thay đổi có thể bị mất. Với CRDT, cả hai thao tác được mô tả thành update độc lập, có định danh và thứ tự logic riêng. Sau khi Liveblocks phân phối update tới các client, trạng thái cuối cùng hội tụ thành một nội dung chứa cả hai chỉnh sửa, ví dụ:

Hệ thống môn PTIT phân tán

Điểm quan trọng là các client không cần nhận update theo cùng một thứ tự tuyệt đối. Miễn là nhận đủ update hợp lệ, chúng sẽ tiến tới trạng thái tương đương.

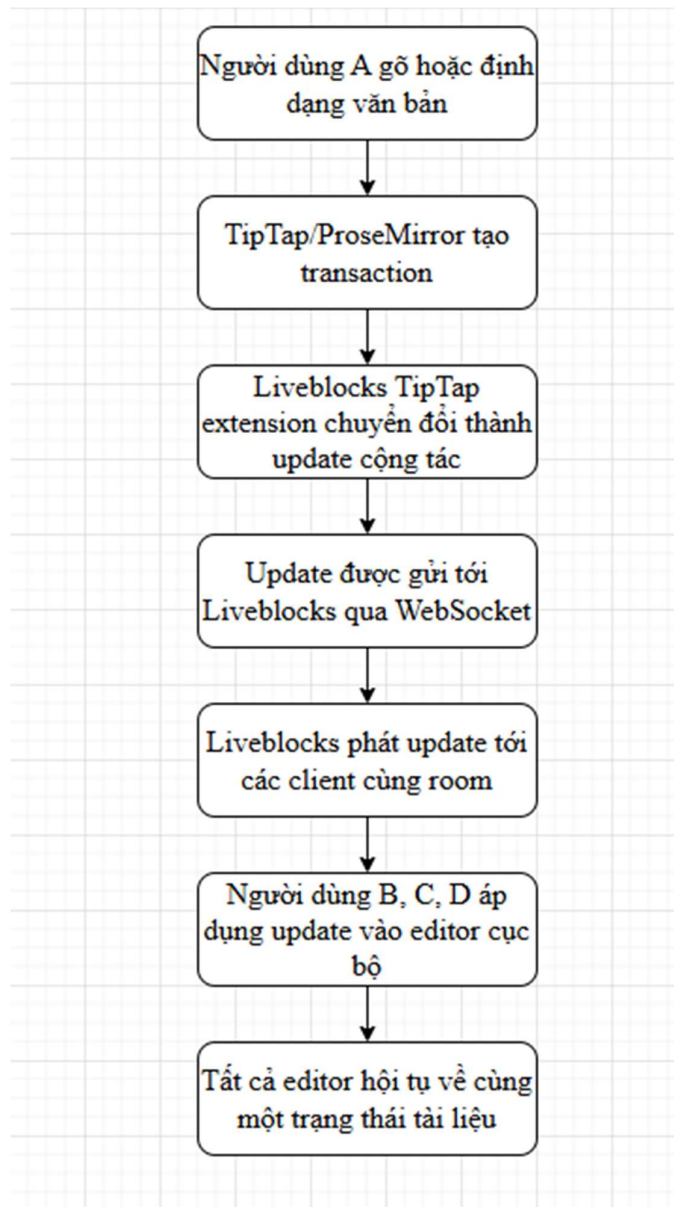
2.4. Giao thức đồng bộ

Liveblocks sử dụng kết nối WebSocket để trao đổi update giữa client và cloud realtime. Ở phía editor, TipTap tạo transaction khi người dùng gõ phím, xóa, định dạng hoặc thao tác trên node. Liveblocks TipTap extension chuyển transaction đó thành update cộng tác và gửi qua room.

Các nhóm message chính trong hệ thống:

Nhóm message	Nội dung	Tác động
Editor update	Thay đổi cấu trúc rich text	Đồng bộ nội dung tài liệu
Presence update	User online, avatar, màu, vị trí con trỏ	Hiển thị cộng tác trực quan
Storage update	Lề trái, lề phải và trạng thái dùng chung	Đồng bộ cấu hình tài liệu
Comment/thread event	Bình luận, phản hồi, trạng thái thread	Trao đổi trực tiếp trên tài liệu
Auth/session event	Cấp quyền room qua API route	Bảo vệ kết nối realtime

2.5. Trình tự đồng bộ thời gian thực đầu-cuối



Hình 3. Luồng đồng bộ nội dung editor.

Trong quá trình này, editor phía người gửi văn bản phản hồi ngay vì thao tác được áp dụng cục bộ trước. Các client còn lại nhận update qua mạng và cập nhật giao diện gần như tức thời.

2.6. Lưu trữ và khôi phục trạng thái

Hệ thống tách metadata và nội dung cộng tác. Convex lưu bản ghi tài liệu và các thông tin phục vụ danh sách/tìm kiếm; Liveblocks giữ trạng thái realtime của room.

2.6.1. Luồng đọc khi mở tài liệu

1. Người dùng mở `/documents/[documentId]`.
2. Server lấy token Clerk cho Convex.

3. Convex query *documents.getById* để kiểm tra tài liệu và metadata.
4. Client render *Document*.
5. RoomProvider mở room Liveblocks có id tương ứng với *documentId*.
6. Liveblocks đồng bộ nội dung hiện tại về TipTap editor.

2.6.2. Luồng ghi khi chỉnh sửa

1. Người dùng chỉnh sửa trong TipTap.
2. Transaction được áp dụng cục bộ.
3. Liveblocks gửi update realtime tới room.
4. Các client khác nhận và áp dụng update.
5. Metadata như đổi tên tài liệu được ghi qua Convex mutation *documents.updateById*.

Điểm khác với mô hình REST thuần là nội dung rich text không phải gửi toàn bộ document qua HTTP sau mỗi lần gõ. Hệ thống chỉ trao đổi update cộng tác nhỏ hơn và phù hợp với tần suất cao.

2.7. Đồng bộ room storage

Ngoài nội dung văn bản, dự án còn đồng bộ một số trạng thái dùng chung trong room, ví dụ *leftMargin* và *rightMargin*. Các giá trị này giúp ruler và layout tài liệu nhất quán giữa những người cùng mở tài liệu.

Room storage phù hợp với dữ liệu nhỏ, cần chia sẻ theo tài liệu và thay đổi không quá phức tạp. Metadata lớn hoặc cần truy vấn danh sách vẫn nên đặt ở Convex.

2.8. Đồng bộ presence, cursor và comments

Presence cho biết ai đang online trong room, màu đại diện và trạng thái tương tác. Collaboration cursor hiển thị vùng chọn hoặc con trỏ của người khác trong editor. Comments/threads cho phép thảo luận theo vị trí nội dung.

Ba nhóm dữ liệu này không nên lưu như metadata tài liệu thông thường:

- Presence thay đổi liên tục và có tính tạm thời.
- Cursor/selection phụ thuộc phiên làm việc hiện tại.
- Comments cần gắn với ngữ cảnh realtime của tài liệu.

Liveblocks cung cấp sẵn UI và API cho các dữ liệu này, giúp giảm độ phức tạp khi xây dựng trải nghiệm cộng tác.

2.9. Tính nhất quán cuối và các đảm bảo của hệ thống

Hệ thống hướng tới tính nhất quán cuối cho nội dung cộng tác: khi các client còn kết nối và nhận đủ update, chúng sẽ hội tụ về cùng một trạng thái. Người dùng có thể thấy thay đổi cục bộ ngay lập tức, sau đó các client khác được cập nhật qua WebSocket.

Các đảm bảo chính:

- Không cần khóa toàn bộ tài liệu khi nhiều người cùng sửa.
- Không ghi đè toàn bộ nội dung sau mỗi thao tác nhỏ.

- Phạm vi đồng bộ được cô lập theo room, tương ứng từng tài liệu.
- Metadata tài liệu và nội dung realtime được xử lý bằng hai cơ chế phù hợp với đặc tính dữ liệu.

III. GIAO THỨC TRAO ĐỔI DỮ LIỆU

3.1. Giao thức HTTP kết hợp Convex client

3.1.1. Giới thiệu và định nghĩa

HTTP là nền tảng trao đổi request/response giữa client và server. Trong dự án, client không gọi REST endpoint tự viết cho mọi thao tác dữ liệu mà sử dụng Convex client. Về bản chất, đây vẫn là luồng trao đổi qua mạng theo mô hình request/response và subscription cho query/mutation.

3.1.2. Cách thức áp dụng trong dự án

Các thao tác quản lý tài liệu nằm trong *convex/documents.ts*:

API	Loại	Chức năng
documents.create	Mutation	Tạo tài liệu mới
documents.get	Query	Lấy danh sách tài liệu, hỗ trợ tìm kiếm và phân trang
documents.getById	Query	Lấy chi tiết một tài liệu
documents.updateById	Mutation	Đổi tên tài liệu
documents.removeById	Mutation	Xóa tài liệu
documents.getByIds	Query	Lấy nhiều tài liệu theo id

Client gọi các API này thông qua Convex React hooks. Khi cần xác thực, Convex nhận thông tin identity từ Clerk.

3.1.3. Định dạng dữ liệu và bảo mật

Dữ liệu trao đổi là object có schema rõ ràng theo Convex. Ví dụ bản ghi tài liệu gồm *title*, *ownerId*, *organizationId*, *roomId* và *initialContent*. Các mutation kiểm tra identity trước khi ghi dữ liệu.

Ưu điểm của cách làm này là giảm lỗi do tự thiết kế endpoint và validator thủ công. Nhược điểm là hệ thống phụ thuộc vào runtime và SDK của Convex.

3.1.4. Lý do lựa chọn sử dụng

Convex phù hợp với bài toán danh sách tài liệu vì cung cấp query/mutation, realtime subscription và database serverless trong cùng một nền tảng. Điều này giúp nhóm tập trung vào logic cộng tác thay vì phải tự triển khai backend CRUD, pagination và xác thực token.

3.1.5. Đánh giá ưu và nhược điểm

Tiêu chí	Ưu điểm	Hạn chế
Tốc độ phát triển	Ít mã backend lặp lại, tích hợp React tốt	Phụ thuộc nền tảng Convex
An toàn kiểu dữ liệu	Có schema và generated API	Cần đồng bộ generated files khi schema đổi
Realtime metadata	Query có thể cập nhật tự động	Không thay thế được luồng CRDT tần suất cao
Vận hành	Không cần tự quản trị database server	Khó tùy biến sâu như backend tự viết

3.2. Giao thức WebSocket với Liveblocks

3.2.1. Giới thiệu và định nghĩa

WebSocket là giao thức truyền thông hai chiều trên một kết nối duy trì lâu dài. Khác với HTTP request/response, server có thể chủ động đẩy dữ liệu tới client khi có thay đổi mới.

3.2.2. Cách thức áp dụng trong dự án

Liveblocks dùng WebSocket để kết nối mỗi client tới room của tài liệu. Room id được ánh xạ theo *documentId*. Khi người dùng chỉnh sửa, update được gửi qua WebSocket tới Liveblocks và phát lại cho các client còn lại.

3.2.3. Định dạng dữ liệu và bảo mật

Client không tự tạo quyền truy cập WebSocket bằng secret key. Quyền room được cấp qua */api/liveblocks-auth*. API route này chạy server-side, dùng Clerk session để xác định user, sau đó chuẩn bị session Liveblocks.

3.2.4. Lý do lựa chọn sử dụng

WebSocket phù hợp với editor cộng tác vì luồng dữ liệu có tần suất cao, hai chiều và cần độ trễ thấp. Nếu dùng HTTP polling, client phải liên tục hỏi server, vừa tăng tải vừa tạo độ trễ không cần thiết.

3.2.5. Đánh giá ưu và nhược điểm

Tiêu chí	Ưu điểm	Hạn chế
Độ trễ	Thấp, server đẩy update ngay	Cần quản lý kết nối lâu dài
Tải mạng	Không cần polling liên tục	Dễ bị ảnh hưởng bởi mạng yếu
Trải nghiệm	Phù hợp cursor, presence, editor update	Cần fallback/khôi phục khi mất kết nối
Bảo mật	Có thể cấp quyền theo room	Phải bảo vệ endpoint cấp quyền

3.3. Giao thức đồng bộ CRDT/Yjs

3.3.1. Giới thiệu và định nghĩa

Yjs sync protocol là cơ chế trao đổi update CRDT giữa các bản sao dữ liệu. Mỗi client giữ một bản sao tài liệu; khi có thay đổi, client gửi update thay vì gửi toàn bộ document.

3.3.2. Cách thức áp dụng trong dự án

Dự án sử dụng Liveblocks TipTap extension để tích hợp Yjs/CRDT với editor. TipTap tạo transaction, Liveblocks chuyển thành update cộng tác, sau đó đồng bộ qua room.

3.3.3. Định dạng dữ liệu và bảo mật

Update CRDT thường là dữ liệu nhị phân hoặc message nội bộ của provider. Ở tầng ứng dụng, nhóm không thao tác trực tiếp với format này mà thông qua SDK. Bảo mật tập trung ở bước cấp quyền room và session Liveblocks.

3.3.4. Lý do lựa chọn sử dụng

CRDT phù hợp hơn lock tài liệu vì cho phép nhiều người cùng sửa. Nó cũng phù hợp hơn ghi đè toàn bộ document vì tránh mất dữ liệu và giảm lượng dữ liệu truyền.

3.3.5. Đánh giá ưu và nhược điểm

Tiêu chí	Ưu điểm	Hạn chế
Chỉnh sửa đồng thời	Hợp nhất tự động, không cần khóa	Khó debug hơn request/response
Trải nghiệm người dùng	Phản hồi cục bộ nhanh	Cần provider/editor tích hợp đúng
Mở rộng	Chỉ gửi update thay đổi	Cấu trúc dữ liệu phức tạp
Độ tin cậy	Hội tụ khi nhận đủ update	Cần cơ chế persistence/reconnect tốt

3.4. Giao thức Awareness/Presence

3.4.1. Giới thiệu và định nghĩa

Awareness là lớp dữ liệu tạm thời mô tả trạng thái hiện diện của người dùng: đang online, tên, avatar, màu, con trỏ và vùng chọn. Dữ liệu này thường không cần lưu bền vững như nội dung tài liệu.

3.4.2. Cách thức áp dụng trong dự án

Liveblocks quản lý presence và collaboration cursor. Component `avatars.tsx` hiển thị người đang ở trong room; editor hiển thị con trỏ/vùng chọn của người khác thông qua extension cộng tác.

3.4.3. Định dạng dữ liệu và bảo mật

Presence gắn với session room đã xác thực. Khi người dùng rời room hoặc mất kết nối, trạng thái hiện diện có thể bị xóa hoặc cập nhật theo cơ chế heartbeat của provider.

3.4.4. Lý do lựa chọn sử dụng

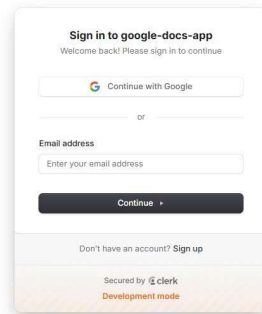
Trong editor cộng tác, người dùng cần biết ai đang xem/sửa tài liệu để tránh thao tác mù. Awareness giúp hệ thống có cảm giác realtime rõ ràng hơn so với chỉ đồng bộ nội dung.

3.4.5. Đánh giá ưu và nhược điểm

Tiêu chí	Ưu điểm	Hạn chế
Trực quan	Thấy người online và vị trí chỉnh sửa	Không phải dữ liệu bền vững
Tương tác nhóm	Giảm xung đột thao tác thủ công	Có thể nhiễu nếu quá nhiều người
Kỹ thuật	Liveblocks hỗ trợ sẵn	Phụ thuộc provider realtime

IV. THỬ NGHIỆM VÀ ĐÁNH GIÁ

4.1. Giao diện đăng nhập và xác thực



Hình 4. Giao diện đăng nhập và xác thực người dùng bằng Clerk

Kịch bản thử nghiệm:

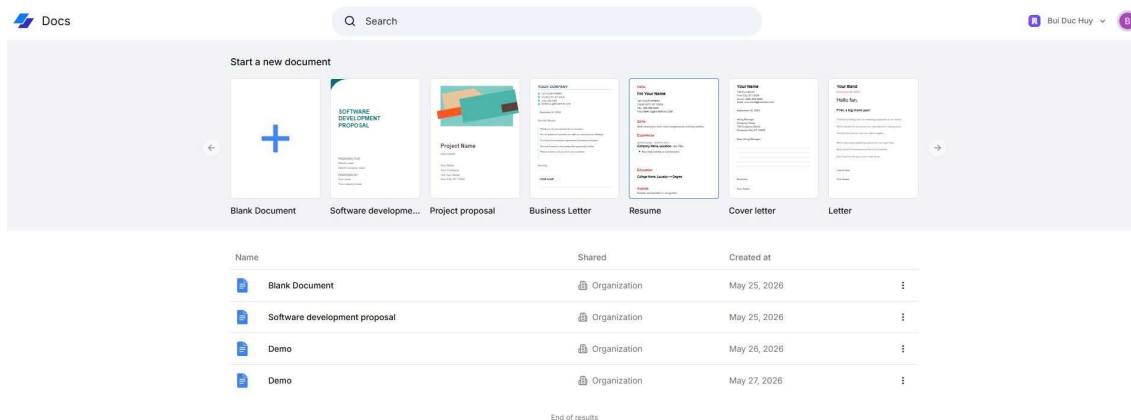
1. Truy cập ứng dụng khi chưa đăng nhập.
2. Kiểm tra form đăng nhập của Clerk xuất hiện.
3. Đăng nhập bằng tài khoản hợp lệ.
4. Kiểm tra ứng dụng chuyển tới trang quản lý tài liệu.
5. Đăng xuất và thử truy cập lại trang tài liệu.

Kết quả mong đợi:

- Người chưa đăng nhập không thao tác được với dữ liệu tài liệu.
- Sau đăng nhập, Convex nhận identity hợp lệ.
- Các API cần quyền truy cập không bị gọi với user rỗng.

4.2. Giao diện trang chủ

Trang chủ hiển thị danh sách tài liệu, ô tìm kiếm và thư viện template. Các component chính gồm navbar, search input, templates gallery, documents table và document menu.



5

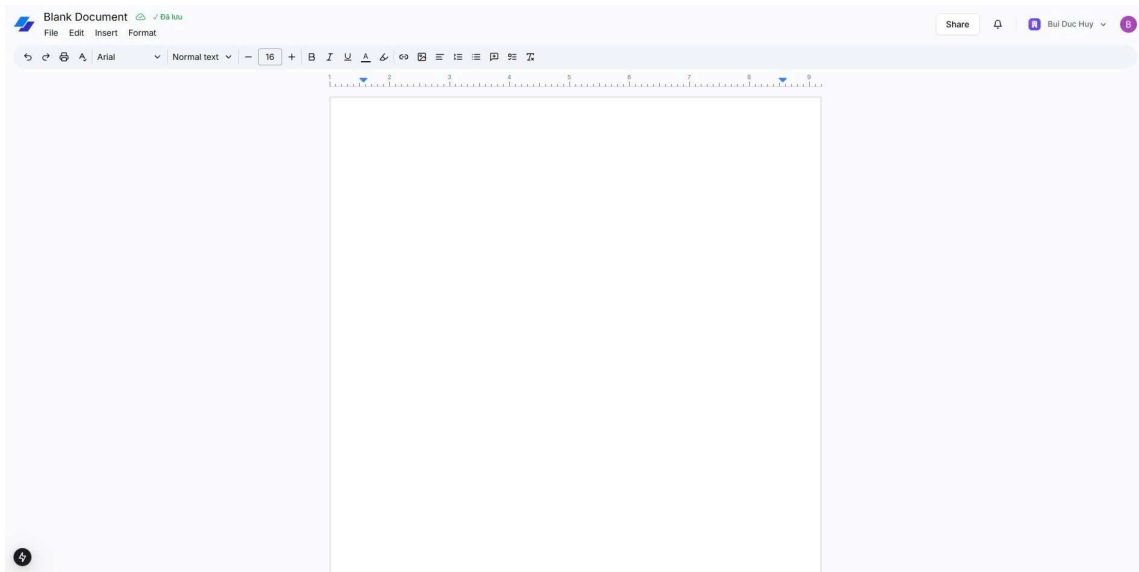
Hình 5. Trang chủ quản lý tài liệu, tìm kiếm và thư viện template
Kịch bản thử nghiệm:

Bước	Thao tác	Kết quả mong đợi
1	Mở trang chủ	Danh sách tài liệu được tải từ Convex
2	Nhập từ khóa tìm kiếm	Danh sách được lọc theo tiêu đề
3	Chọn template	Tạo tài liệu với nội dung khởi tạo tương ứng
4	Mở menu tài liệu	Có thao tác đổi tên/xóa

Đánh giá: chức năng trang chủ đáp ứng nhu cầu quản lý tài liệu cơ bản. Điểm cần lưu ý là các thao tác phải xử lý trạng thái loading và lỗi mạng để tránh người dùng bấm lặp.

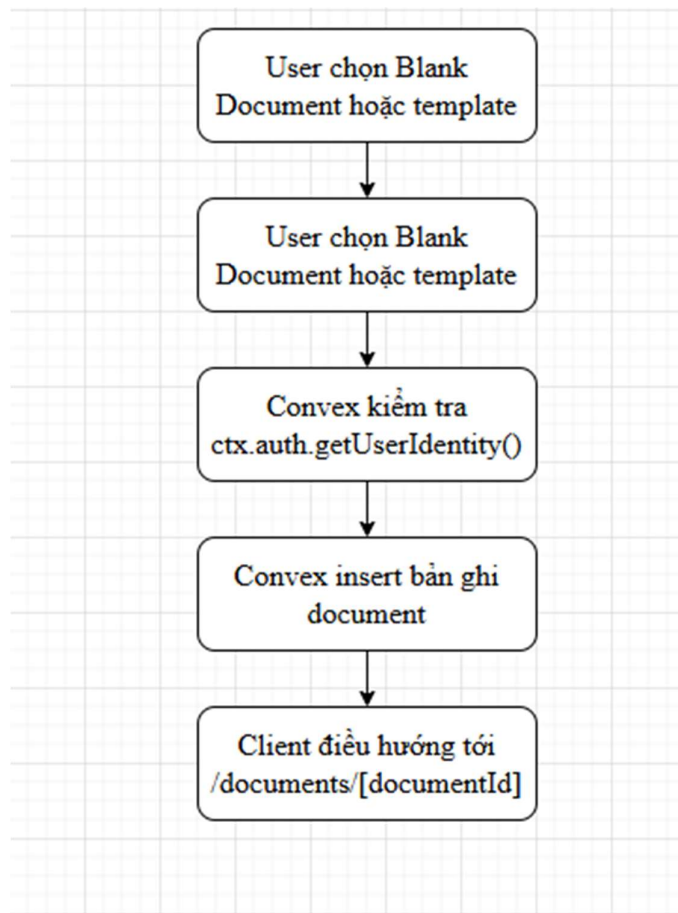
4.3. Tạo và quản lý tài liệu

Khi tạo tài liệu mới, client gọi mutation `documents.create`. Convex kiểm tra identity, lấy `ownerId` và `organizationId` nếu có, sau đó tạo bản ghi trong bảng `documents`.



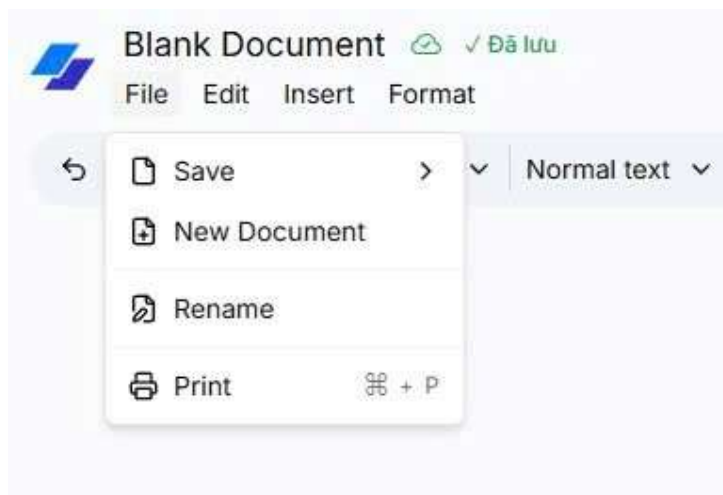
Hình 6. Giao diện soạn thảo tài liệu mới sau khi tạo document

Luồng tạo tài liệu:



Hình 7. Luồng tạo tài liệu mới.

Các thao tác quản lý gồm đổi tên, xóa, tìm kiếm và phân trang. Các chức năng này phù hợp đặt ở Convex vì chúng là nghiệp vụ metadata, không cần WebSocket tần suất cao.



Hình 8. Menu thao tác tài liệu gồm lưu, tạo mới, đổi tên và in tài liệu

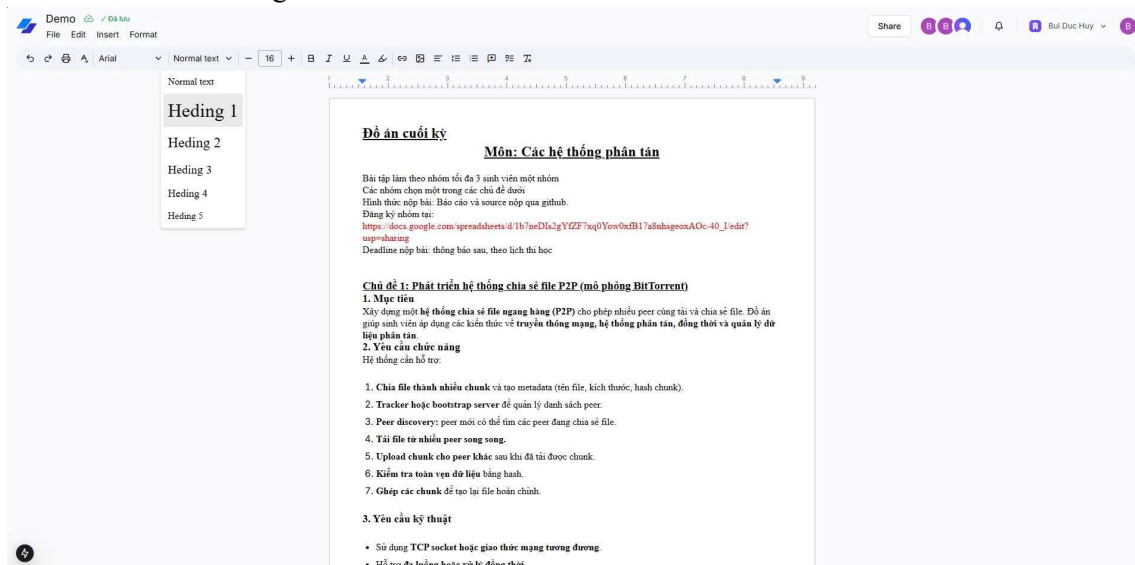
4.4. Soạn thảo rich text



Hình 9. Thanh công cụ định dạng văn bản của trình soạn thảo

Editor hỗ trợ các thao tác văn phòng phổ biến:

- Nhập và xóa văn bản.
- Định dạng bold, italic, underline.
- Heading, paragraph, danh sách số và danh sách gạch đầu dòng.
- Căn trái, căn giữa, căn phải, căn đều.
- Đổi font, cỡ chữ, màu chữ, highlight.
- Chèn bảng, ảnh, link và task list.
- Điều chỉnh lề bằng ruler.



Hình 10. Minh họa soạn thảo rich text với tiêu đề, định dạng văn bản và nội dung tài liệu
Kịch bản kiểm thử:

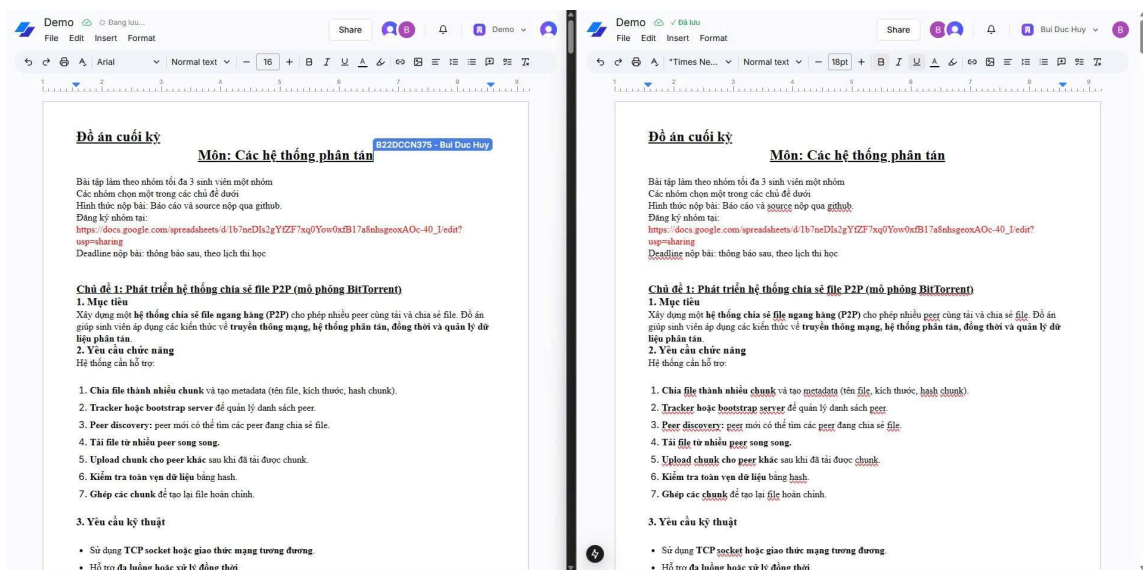
Chức năng	Cách kiểm tra	Kết quả mong đợi
Định dạng chữ	Chọn đoạn văn và bấm toolbar	Định dạng áp dụng đúng vào selection
Heading	Chọn Heading 1/2/3	Node trong editor đổi đúng loại
Bảng	Chèn bảng và nhập dữ liệu	Bảng hiển thị và chỉnh sửa được

Chức năng	Cách kiểm tra	Kết quả mong đợi
Ảnh	Chèn ảnh vào tài liệu	Ảnh xuất hiện trong vùng soạn thảo
Ruler	Thay đổi lề	Lề tài liệu cập nhật theo room storage

4.5. Cộng tác thời gian thực

Kịch bản thử nghiệm chính:

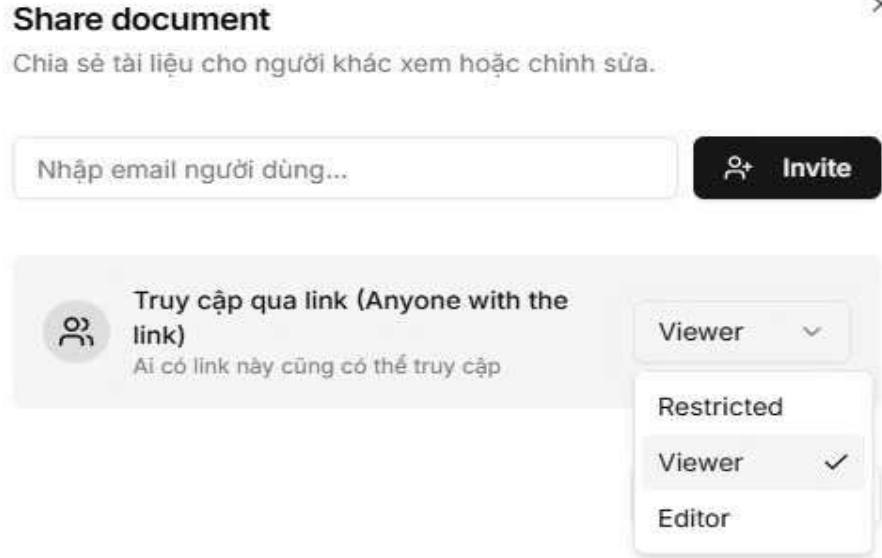
1. Mở cùng một tài liệu ở hai trình duyệt hoặc hai tài khoản khác nhau.
2. Ở client A, nhập một đoạn văn bản.
3. Quan sát client B nhận nội dung mới.
4. Ở client B, định dạng một phần văn bản.
5. Quan sát client A nhận định dạng mới.
6. Hai client cùng nhập vào gần một vị trí để kiểm tra hợp nhất thao tác.



Hình 11. Hai người dùng cùng chỉnh sửa một tài liệu theo thời gian thực.

Kết quả mong đợi:

- Nội dung cập nhật gần như tức thời.
- Không xảy ra mất dữ liệu khi hai người cùng sửa.
- Cursor/presence giúp nhận biết người còn lại đang ở trong tài liệu.
- Room chỉ ảnh hưởng tới tài liệu đang mở, không làm thay đổi tài liệu khác.



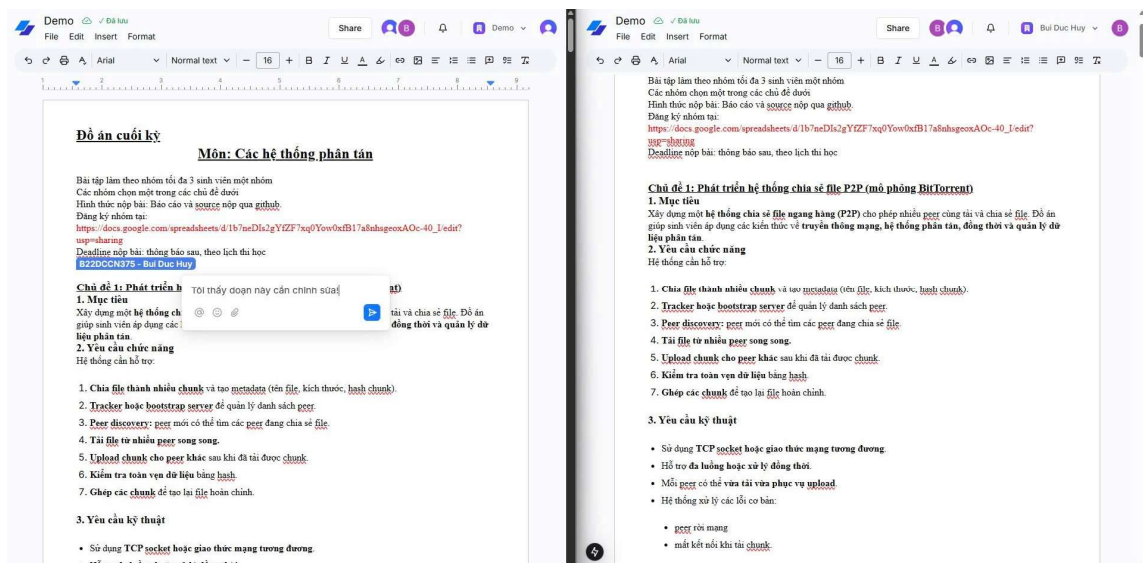
Hình 12. Chức năng chia sẻ tài liệu và phân quyền truy cập qua liên kết

4.6. Bình luận và quản lý người dùng đang chỉnh sửa

Liveblocks hỗ trợ comments và threads để người dùng thảo luận trực tiếp trong tài liệu. Các component `threads.tsx` và `inbox.tsx` phục vụ hiển thị luồng trao đổi, trong khi `avatars.tsx` hiển thị người đang có mặt.



Hình 13. Hiển thị người dùng đang tham gia chỉnh sửa tài liệu



Hình 14. Tạo bình luận trên đoạn văn bản trong môi trường cộng tác thời gian thực

Kịch bản thử nghiệm:

Tính năng	Thao tác	Kết quả mong đợi
Presence	Mở tài liệu bằng hai user	Avatar của user còn lại xuất hiện
Cursor	Chọn hoặc gõ trong editor	Client khác thấy vùng chọn/con trỏ
Comment	Tạo thread trên đoạn văn	Thread hiển thị đúng ngữ cảnh
Inbox	Có comment mới	Người dùng nhận được thông tin trong inbox

Đánh giá: nhóm tính năng này làm rõ tính chất cộng tác của hệ thống, không chỉ là đồng bộ văn bản thuần túy.

4.7. Xuất tài liệu và undo/redo

Dự án có chức năng xuất nội dung tài liệu sang các định dạng như PDF, HTML, JSON hoặc text tùy theo phần cài đặt trong editor/navbar. Đây là chức năng hỗ trợ sau khi người dùng hoàn thành soạn thảo.

Undo/redo trong môi trường cộng tác cần được xử lý cẩn thận. Với editor realtime, lịch sử cục bộ của ProseMirror có thể xung đột với lịch sử cộng tác. Vì vậy, phần StarterKit trong editor được cấu hình theo hướng tương thích với Liveblocks/Yjs, tránh để cơ chế history cục bộ ghi đè hoặc đảo ngược sai thao tác của người khác.

Kịch bản kiểm thử:

1. User A nhập đoạn văn.
2. User B nhập thêm nội dung ở vị trí khác.
3. User A thực hiện undo.
4. Kiểm tra undo chỉ ảnh hưởng hợp lý tới thao tác của A, không xóa mất thao tác của B.
5. Thử redo và quan sát hai client hội tụ lại trạng thái giống nhau.

4.8. Đánh giá chung

Tiêu chí	Đánh giá
Kiến trúc	Phân tách tốt giữa UI, metadata, realtime và xác thực
Đồng bộ	Phù hợp bài toán cộng tác nhờ Liveblocks/Yjs và CRDT
Bảo mật	Secret key đặt server-side, Convex kiểm tra identity
Khả năng mở rộng	Dựa vào dịch vụ cloud/serverless, giảm gánh nặng vận hành
Trải nghiệm	Có editor rich text, presence, comments, template và quản lý tài liệu
Hạn chế	Phụ thuộc dịch vụ bên ngoài; cần kiểm thử kỹ khi mất mạng hoặc nhiều user cùng sửa

4.9. Hướng phát triển

Các hướng mở rộng phù hợp:

- Bổ sung phân quyền chi tiết theo vai trò OWNER, EDITOR và VIEWER ở cấp tài liệu.
- Thêm chia sẻ tài liệu qua email hoặc link công khai có giới hạn quyền.
- Xây dựng lịch sử phiên bản và khôi phục snapshot.
- Cải thiện chế độ offline và thông báo trạng thái kết nối.
- Bổ sung kiểm thử tự động cho Convex functions, API route và luồng editor.
- Thêm dashboard quản trị organization và thống kê hoạt động tài liệu.

KẾT LUẬN

Báo cáo đã trình bày cách xây dựng hệ thống soạn thảo văn bản cộng tác thời gian thực theo góc nhìn hệ phân tán. Hệ thống phân chia rõ trách nhiệm giữa Next.js/React ở tầng client, Convex ở tầng dữ liệu nghiệp vụ, Liveblocks/Yjs ở tầng đồng bộ CRDT và Clerk ở tầng xác thực.

Với mô hình một tài liệu tương ứng một realtime room, các thao tác chỉnh sửa được cô lập theo phạm vi tài liệu và có thể đồng bộ tới nhiều client. Việc sử dụng CRDT giúp hệ thống xử lý chỉnh sửa đồng thời mà không cần khóa toàn bộ tài liệu. Các giao thức HTTP/Convex và WebSocket/Liveblocks bổ trợ nhau: HTTP phù hợp với nghiệp vụ quản lý tài liệu, còn WebSocket phù hợp với luồng cộng tác tần suất cao.

Hệ thống hiện đáp ứng các yêu cầu chính của bài toán: đăng nhập, quản lý tài liệu, soạn thảo rich text, đồng bộ thời gian thực, presence, cursor và bình luận. Trong tương lai, dự án có thể hoàn thiện thêm phân quyền chia sẻ, lịch sử phiên bản và khả năng hoạt động ngoại tuyến để tiến gần hơn tới một trình soạn thảo cộng tác hoàn chỉnh.